

BUILD BRIEF

Huemen.studio — v1

What we are building, how it is structured, and what “done” means for each module. Internal reference for the build team — not for client circulation.

Product	Huemen.studio — multi-tenant SaaS personal-brand OS, white-labelled to The Brand Professor
Client	Sahil Gandhi / The Brand Professor (UK & India)
Build window	3 weeks from kick-off, weekly preview at end of each week
End state	Full IP and source handed to client; accounts in client's name
Status	v1.0 — living document, update in repo as decisions land

01 — OBJECTIVE

What we are actually building

A multi-tenant web app where a personal brand is defined once and everything published from it — copy and images — is generated from that stored context. The client (Sahil) runs it as his own product; his coaching and workshop clients each get an isolated workspace inside it.

The whole product rests on one idea: brand context is stored server-side and injected into every generation, so nobody re-briefs an AI tool from scratch. If we get context storage and injection right, the rest is CRUD and UI. If we get it wrong, we have built another wrapper.

THREE THINGS THAT MUST BE TRUE AT HANDOVER

- A client can log in, fill their brand foundation once, and produce an on-brand LinkedIn post and its image without typing any brand context into a prompt.
- Two tenants cannot see each other's data under any query path. This is the one bug class we cannot ship.
- Sahil's team can create, provision and monitor client workspaces without us touching the database.

02 — USERS AND ROLES

Who is in the system

ROLE	WHO	CAN DO
Owner / Admin	Sahil and his team	Create and archive client workspaces, invite users, view all workspaces, manage branding and global prompt templates, view usage analytics

ROLE	WHO	CAN DO
Coach	Sahil's team members	Access assigned client workspaces, edit brand foundation and content on the client's behalf, cannot manage billing-level settings or other coaches' clients
Client	Workshop / cohort / 1:1 clients	Access their own workspace only — brand foundation, content studio, visual studio, calendar, idea inbox, offers, exports

Client accounts are admin-provisioned in v1 — there is no public sign-up. This is deliberate: it removes onboarding flows, email verification edge cases and abuse handling from a three-week build, and it matches how Sahil actually sells (a client joins after a workshop, not off a landing page).

03 — DATA MODEL

Core objects

Every table below carries a `tenant_id`. No exceptions, including join tables and logs. Tenant scoping is enforced at the database layer, not in application code — see section 07.

TABLE	HOLDS
<code>tenants</code>	Workspace record: name, custom domain, branding JSON (palette, logo refs, fonts), status
<code>users</code>	Email, role, <code>tenant_id</code> , invite/active status, last seen
<code>brand_profiles</code>	Story arc (3 chapters), positioning statement, niche, audience definition, offers
<code>voice_guides</code>	Tone descriptors, do-words, don't-words, sample posts, reading level, formatting rules
<code>visual_identities</code>	Palette hex values, fonts, logo asset ref, image style notes, aspect-ratio defaults
<code>assets</code>	Uploaded files — logos, fonts, reference images. Storage key, mime, dimensions, meta
<code>pillars</code>	Content pillars per brand, used to tag ideas and calendar entries
<code>content_items</code>	Generated or edited copy: channel, format, hook, body, CTA, pillar, status, <code>prompt_version</code>
<code>image_assets</code>	Generated visuals: parent <code>content_item</code> , prompt used, model, params, aspect ratio, storage key
<code>ideas</code>	Idea inbox entries: raw text, source, auto-tagged pillar, <code>converted_to_content_item_id</code>
<code>calendar_entries</code>	Scheduled slot: date, pillar, channel, linked <code>content_item</code> , status
<code>offers</code>	Offer designer output: name, format, promise, deliverables, pricing logic notes

TABLE	HOLDS
prompt_templates	Versioned prompt bodies by key, target model, params. Global and per-tenant overrides
generation_logs	Every model call: kind, model, tokens in/out, image count, latency, status, user, tenant

The brand context object. brand_profiles + voice_guides + visual_identities compose into a single serialised context block that is injected into every generation. Build this as one function with one output shape, versioned. Every module calls it. Do not let modules assemble their own context — that is how voice drift starts.

04 — MODULES

What to build, module by module

4.1 Brand Foundation

- Guided editor for story arc (3 chapters), positioning statement, niche, audience, offers.
- Voice guide capture: tone sliders or descriptors, do/don't word lists, 3–5 sample posts pasted by the client.
- Visual identity capture: palette hex inputs, font selection or upload, logo upload, image style notes, reference image uploads.
- Pre-workshop questionnaire: a public tokenised form that writes straight into a draft brand profile, so the workshop itself becomes onboarding. Post-workshop summary can be pasted in and parsed into fields.
- Completeness indicator — the content studio should visibly degrade if the foundation is thin, and say so, rather than generating generic output silently.

4.2 AI Content Studio (text)

- Formats: LinkedIn post, LinkedIn hook set, Instagram caption, Instagram carousel copy (slide-by-slide), newsletter section, pitch email, talk abstract.
- Generation flow: pick format → optional topic or idea → context injected automatically → 2–3 variants returned → edit inline → save as content_item.
- Voice guardrails: do/don't words enforced in the prompt and checked post-generation; flag violations in the UI rather than silently rewriting.
- Regenerate with a steer (“shorter”, “more contrarian”, “less corporate”) without losing the previous version — keep generation history per content_item.
- 90-day calendar generator: takes pillars + posting frequency, returns dated slots with topic, hook angle and CTA, written into calendar_entries as drafts.

4.3 AI Visual Studio (images)

- Sits in the same screen as the copy — generate the post, then generate its visual without navigating away.
- Output types: single-image LinkedIn/Instagram post, carousel frame set, quote card, announcement graphic.
- Brand-aware prompting: palette, type treatment and style notes from visual_identities are appended to every image prompt, with negative prompts for the generic AI look.
- **Decision needed (see section 09):** pure model generation vs. templated canvas with model-generated backgrounds and real text overlay. Templated is far more reliable for carousels and quote cards, because

text rendering in image models is still unreliable and misspelt text on a client's LinkedIn carousel is a brand-damaging failure, not a cosmetic one.

- Regenerate, keep variants, download at correct export dimensions per channel.

4.4 Strategy and packaging tools

- Niche validation prompt flow; 3×3 positioning matrix; authority ladder; Q1–Q4 growth plan generator.
- Offer designer: expertise in, packaged offer out — format, promise, deliverables, pricing logic. Saves to offers, reusable as context for pitch emails.
- Answer-first (AEO) page generator: structured Q&A pages built from the brand profile, exportable as HTML or Markdown.

4.5 Workflow and collaboration

- Idea inbox: quick capture (text paste, later email-in), auto-tagged to a pillar, one-click convert to a content_item.
- Repurposing pipeline: take one long-form item and fan it out to short posts, carousel copy, an email and a talk outline, all linked back to the source.
- Client portal shell: navigation, workspace switcher for coaches, per-client isolation enforced in the data layer.
- Export: PDF and Markdown/Notion-compatible export of brand foundation, calendar and content.

4.6 Admin and analytics

- Workspace management: create, invite, archive. Assign coaches to clients.
- Usage view per tenant: generations run, images produced, active users, last activity. Reads from generation_logs.
- Per-tenant usage caps with a soft warning and a hard stop, configurable by admin. The client pays provider costs directly, so runaway usage on one workspace is his problem to see and control — surface it, don't hide it.
- Global prompt template management with versioning, so prompts can be improved without a redeploy.

4.7 White-label layer

- Custom domain support with TLS; branding config (logo, palette, fonts) applied to the client-facing UI and transactional emails.
- No Okra Tech Labs marks anywhere in the client-facing experience — check email templates, page titles, favicons, error pages and PDF exports specifically.

05 — ARCHITECTURE

Stack and shape

Chosen for speed of delivery and for being handover-friendly — the client will own this codebase, so nothing exotic and nothing that needs us to run it.

LAYER	APPROACH
Frontend	Next.js (App Router) + TypeScript, Tailwind, component library. Server actions for mutations, streaming for generation output

LAYER	APPROACH
Backend	Next.js API routes / server actions. Long-running generation (carousels, calendars) goes to a background job queue, not a request thread
Database	Postgres with row-level security for tenant isolation. Migrations checked into the repo
Auth	Managed auth provider with organisation/tenant support and role claims. Email invite flow only in v1
Storage	Object storage for uploads and generated images, signed URLs, tenant-prefixed keys
AI layer	Provider-agnostic wrapper: one interface for text, one for image. Model choice and params live in config, never hardcoded in feature code
Jobs	Queue for batch generation, retries with backoff, dead-letter handling and visible job status in the UI
Observability	Structured logs, error tracking, generation_logs as the product-level audit trail

THE AI LAYER, SPECIFICALLY

- One context builder function, versioned. Every prompt is assembled as: system template (from prompt_templates) + brand context block + task input.
- Prompt templates are data, not code. Versioned, editable by admin, and the version used is stamped on every content_item so we can trace a bad output back to a prompt.
- Streaming responses to the UI for anything the user waits on; queued jobs for anything over roughly ten seconds.
- Every call writes a generation_log row before returning. No silent failures — a failed generation is a visible, retryable state in the UI.
- Retry and fallback policy defined once at the wrapper level, so a provider hiccup does not surface as a broken screen.

06 — BUILD SEQUENCE

Three weeks

Week boundaries are preview points, not hard gates — but anything not started by its week is a scope conversation, not an overtime problem.

WEEK	SHIPS
1	Schema and migrations with RLS in place from the first table. Auth, roles, tenant provisioning, admin shell. Brand Foundation editor end to end, including asset upload and the questionnaire intake form. Context builder function written and unit-tested.
2	AI wrapper and prompt template store. Content Studio: all text formats, variants, inline editing, generation history, voice guardrail checks. Visual Studio: image generation with brand-aware prompting, variant handling, export dimensions. Client portal shell with isolation verified.

WEEK	SHIPS
3	Calendar generator, idea inbox, offer designer, repurposing pipeline, AEO pages. Exports. Admin usage analytics and caps. Custom domain, full white-label pass. Security review, seed data, documentation, repo and account handover.

DEPENDENCIES WE DO NOT CONTROL

- Client brand assets — palette, logos, fonts, reference posts and visuals — are due on day one. Week two is blocked on these, not week three: the Visual Studio cannot be tuned without them.
- Domain and provider accounts in the client's name are needed by the start of week three for the white-label pass.
- Voice sign-off from Sahil in week two. Book this in advance; it is the only review that cannot be deferred.

07 — NON-FUNCTIONAL REQUIREMENTS

Isolation, security, performance

Tenant isolation

- Row-level security policies on every table, keyed to the session tenant claim. Application-layer filtering is a second line of defence, never the only one.
- Storage keys prefixed by tenant; signed URLs scoped and short-lived.
- Automated tests that attempt cross-tenant reads on every endpoint and expect failure. These run in CI and block merge.

Security

- Secrets in environment config only, never in the repo. Rotate before handover.
- Uploads: type and size validation, image re-encoding, no direct execution paths.
- Rate limiting per user and per tenant on generation endpoints.
- Audit trail on admin actions — workspace creation, role changes, cap changes.
- Client data is never used for model training or evaluation. This is a contractual obligation, not a preference — configure providers accordingly and record the setting.

Performance and reliability

- First token visible within a few seconds for single-item text generation; anything longer streams or queues.
- Batch operations (carousel sets, 90-day calendars) always queue, with progress visible and partial results retained on failure.
- Graceful degradation when a provider is down: clear message, retry, saved input. Never lose a user's draft.

08 — SCOPE BOUNDARY

Not in v1

These are deferred, not cancelled. If any of them enters a conversation mid-build, it goes on the phase-two list and the client is told the same day — not absorbed.

NOT BUILDING	WHY
Video generation or editing	Explicitly out per discovery. Launch surfaces are LinkedIn and Instagram, both carried by text and stills
Direct publishing to LinkedIn / Instagram APIs	Review, permissions and token maintenance would consume most of a week. Export and copy-out in v1
Client self-serve sign-up and subscription billing	Accounts are admin-provisioned; the client bills through his existing coaching relationship
Native mobile apps	Responsive web only
Paid-ads creative, landing pages, CRM	Not part of the personal-brand loop we are building
Analytics on published post performance	Requires platform API access. Usage analytics only in v1

09 — OPEN DECISIONS

Settle these in week one

DECISION	NOTES AND DEFAULT
Carousel and quote-card rendering	Templated canvas with real text overlay and model-generated backgrounds, vs. pure image generation. Default: templated. Image models still misrender text, and a typo on a client's carousel is a brand failure we own
Image model choice	Evaluate on brand-consistency and aspect-ratio control, not on benchmark quality. Pick in week one so week two is build, not evaluation
Text model routing	One strong model for drafting vs. a cheaper model for short tasks like hooks and tags. Default: route by task, configured, not hardcoded
Fonts in generated images	Brand fonts may not be licensed for embedding or server-side rendering. Check licences when assets arrive on day one — this can force a fallback
Export format for Notion	Markdown export vs. Notion API integration. Default: Markdown, no integration in v1
Multi-language support	Sahil operates across the UK and India. Confirm whether v1 is English-only. Default: English-only, with the context object structured so language is a parameter later

10 — RISKS

What is most likely to go wrong

RISK	HANDLING
Generated images look generic and off-brand	Highest-probability failure and the one the client will judge us on. Tune prompting against real reference assets in week two; templated rendering as the fallback path

RISK	HANDLING
Assets arrive late	Week two is blocked, not week three. Escalate on day two if nothing has landed; timeline shifts day for day, and that is already agreed in writing
Voice output feels generic to Sahil	He is a brand professional judging voice quality — a low bar will not pass. Get sample posts early and sign off voice in week two, not week three
Cross-tenant data leak	The one unshippable bug. RLS from day one plus CI tests that actively attempt cross-tenant access
Scope creep during weekly previews	Previews invite ideas. Capture everything, commit to nothing in the room, respond same day with in-scope or phase-two
Handover friction at the end	Accounts in the client's name from week three, not on the last day. Documentation written as we go, not retrofitted

11 — DEFINITION OF DONE

Handover checklist

The build is complete when every line below is true, not when the last feature merges.

PRODUCT

- ☐ A new client workspace can be created, invited and used end to end without developer involvement.
- ☐ Brand foundation, content studio, visual studio, calendar, idea inbox and offer designer all work against a real brand profile with real assets.
- ☐ Generated copy and images pass Sahil's voice and brand review.
- ☐ Exports produce clean PDF and Markdown output with client branding.

ENGINEERING

- ☐ Cross-tenant access tests pass in CI on every endpoint.
- ☐ No Okra Tech Labs marks in any client-facing surface, including emails, exports and error pages.
- ☐ Secrets rotated; no credentials in the repo or its history.
- ☐ Migrations run clean from empty on a fresh database.

HANDOVER

- ☐ Source code in the client's own repository, with full history.
- ☐ Hosting, database, storage, domain and model provider accounts in the client's name, with access transferred and our access removed on confirmation.
- ☐ README covering local setup, environment variables, deployment, and how to edit prompt templates.
- ☐ Architecture note and data model diagram checked into the repo.
- ☐ Phase-two list written up from everything parked during the build, handed over with the code.

Internal document — Okra Tech Labs. Not for client circulation. Commercial terms are covered separately in the proposal and statement of work.